APP GUIDE · V0.1

FTGU PRODUCTION BOARD

One board for the whole YouTube studio. Every role gets its own nav, columns and capabilities. Postgres decides who can change what.

SCRIPT TO LIVE, ONE BOARD



PRODUCT	STACK	READ TIME	PREPARED BY
FTGU Studio	Next.js 16, Supabase	About 12 min	ZZMEDIA Studio

CONTENTS

WHAT THIS COVERS

01	**Overview**	What the app is and what it runs on	03
02	**Architecture**	How the pieces connect	04
03	**Roles**	Four user types, four boards	05
04	**Pipeline**	Stages, statuses and workflow rules	06
05	**Pages**	Navigation by role	07
06	**Access**	Where permissions live	08
07	**Invites**	How people join the workspace	09
08	**Integrations**	Ten connected services	10
09	**AI and research**	Assistant tools and the research agent	11
10	**Run it**	Setup, environment and roadmap	12

> HOW TO READ THIS
>
> One page per topic. Mono text names real files, routes and tables in the repo.

01 START HERE

WHAT IT IS

A production board for the FTGU YouTube studio, with a separate view for every person who touches a video.

4	5	10	42
USER ROLES	BOARD VIEWS	INTEGRATIONS	TABLES

What it does

Tracks every video from script to live

Film, edit, thumbnail, sponsor and publish dates on one row.

Gives each role its own board

Editors see an edit queue. Designers see a design queue. Sponsors see their account and approve from it.

Finds and drafts ideas

A research agent scans YouTube, TikTok, Reddit and Google, then drafts ideas.

Answers from the board

An AI assistant searches, plans and creates projects inside your permissions.

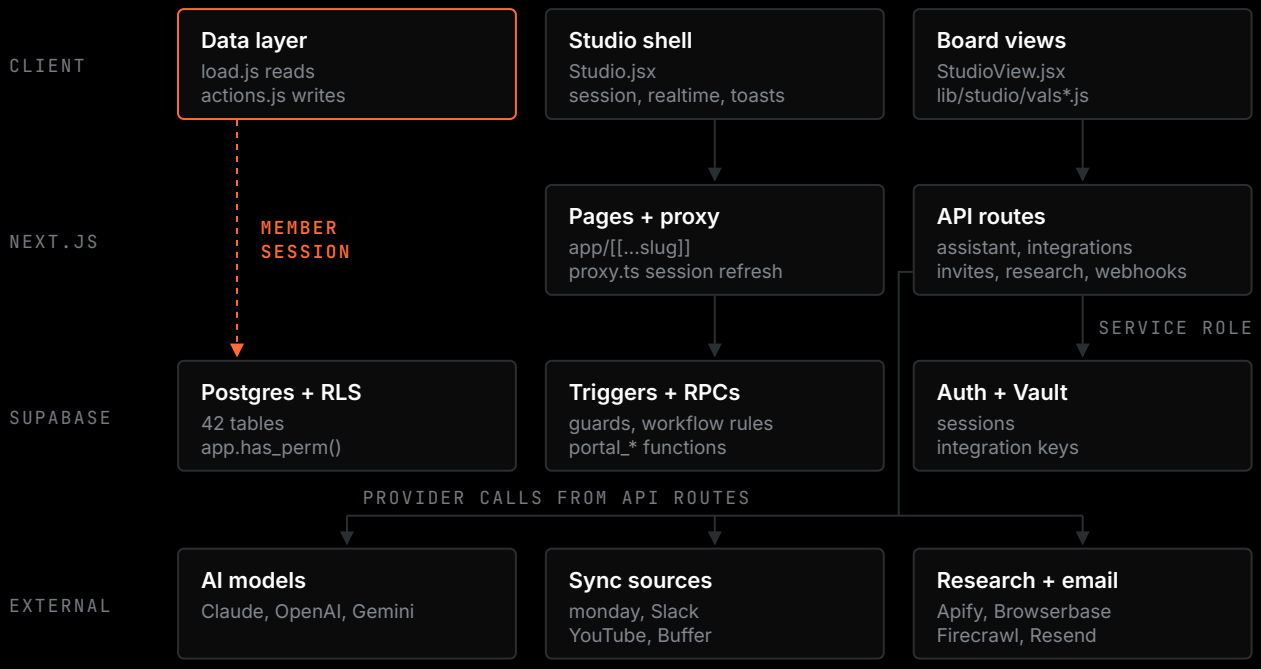
Stack

LAYER	TECHNOLOGY
App	Next.js 16.4, React 19.3, Tailwind 4, TypeScript
Data	Supabase Postgres, Auth, Vault, Realtime
AI	Vercel AI SDK 7 with Anthropic, OpenAI, Google
Email	Resend
Hosting	AWS Amplify, Node 22, pnpm

02 SYSTEM

HOW IT CONNECTS

The browser reads and writes Supabase on the member's own session. Server routes handle anything that needs a secret.



PATH	HOLDS
`supabase/migrations/`	Schema, RLS, triggers, portal RPCs, defaults
`components/studio/`	Shell and design markup as JSX
`lib/studio/`	View logic, reads and every write
`lib/integrations/`	Provider registry, Vault access, one file per provider
`lib/ai/`	Model picker, system prompt, assistant tools
`app/api/`	Assistant, integrations, invites, research, webhooks

03 WHO SEES WHAT

FOUR BOARDS

The signed-in member's role sets nav, columns, views and scope. No preview switcher.

ROLE	BOARD	VIEWS	SCOPE
Super Admin	Full board	Table, Kanban, List, Timeline, Calendar	All projects
Editor	Edit queue	Table, Kanban, List, Timeline	Assigned
Designer	Design queue	Table, Kanban, Calendar	Assigned
Sponsor / Client	Account view	Table, Calendar	Own account

Columns by role

	PRODUCTION					EDIT			DESIGN			SPONSOR		PUBLISH			
	Project	Job #	Channel	Type	Film date	Editor	Edit status	Edit due	Designer	Thumbnail	Thumb due	Sponsor	Approval	Status	Pub date	Distribution	URL
Super Admin	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●
Editor	●	●	●	●	○	●	●	●	○	○	○	○	○	○	●	○	●
Designer	●	●	●	●	○	○	○	○	●	●	●	○	○	○	●	○	○
Sponsor / Client	●	○	○	○	○	○	○	○	○	○	○	○	○	●	●	○	●

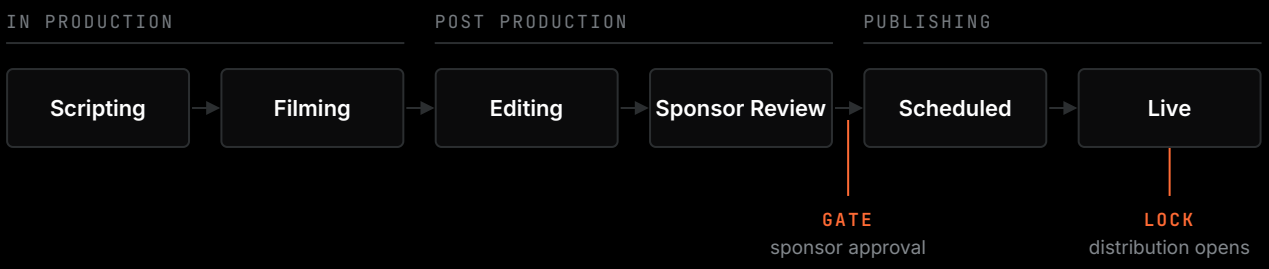
Kanban card by role

ROLE	BADGES	DATES	COLOR BAR
Super Admin	Status, edit, thumbnail	Film, Pub	Status
Editor	Editor status	Edit due, Pub	Editor status
Designer	Thumbnail	Thumb due, Pub	Thumbnail
Sponsor / Client	Status	Pub	Status

04 WORKFLOW

SCRIPT TO LIVE

Six statuses across three stages. Triggers enforce the gates, so no one skips a step.



Rules enforced in Postgres

Sponsor approval before Scheduled

A sponsored video cannot move to Scheduled until the sponsor approves.

Locked after Live

Published projects stop accepting edits.

Distribution on Live

Clipping, Newsletter, YT Community and Distribute tasks open automatically.

Own fields only

Editors and designers change their own status fields and nothing else.

Status fields

FIELD	VALUES
Editor status	Assigned, Editing, In Review, Needs Revision, Approved
Thumbnail	Not Started, In Design, In Review, Needs Revision, Done
Sponsor approval	Pending, Approved, Changes Requested
Distribution	Not Yet, Ongoing, Drafted, Done

05 NAVIGATION

PAGES BY ROLE

Super Admins get the full workspace.
Everyone else gets only what their job needs.

PAGE	WHAT IT HOLDS	SA	ED	DE	SP
Daily	AI assistant and the day at a glance	●	○	○	○
Projects	The board: table, kanban, list, timeline, calendar	●	●	●	●
Ideas & Research	Ideas, scripts and research drafts	●	○	●	●
Brand	Brand kits and shared files	●	●	●	●
Inbox	Notifications, activity and Slack messages	●	●	●	●
Content	YouTube videos and Buffer posts with metrics	●	○	○	○
Users	Invite, resend, archive, make Super Admin	●	○	○	○
Roles & Permissions	Permission grid per role	●	○	○	○
Workload	Capacity and workflow rules	●	○	○	○
Clients & Sponsors	Accounts, sponsorships, approvals	●	○	○	○
Finance	Deals, invoices, payouts	●	○	○	○
Automations	Workflows, connector status	●	○	○	○
Activity Log	Audit trail of every change	●	○	○	○
Workspace Settings	Integrations, AI models, defaults	●	○	○	○
Settings	Profile, password, preferences	○	●	●	●

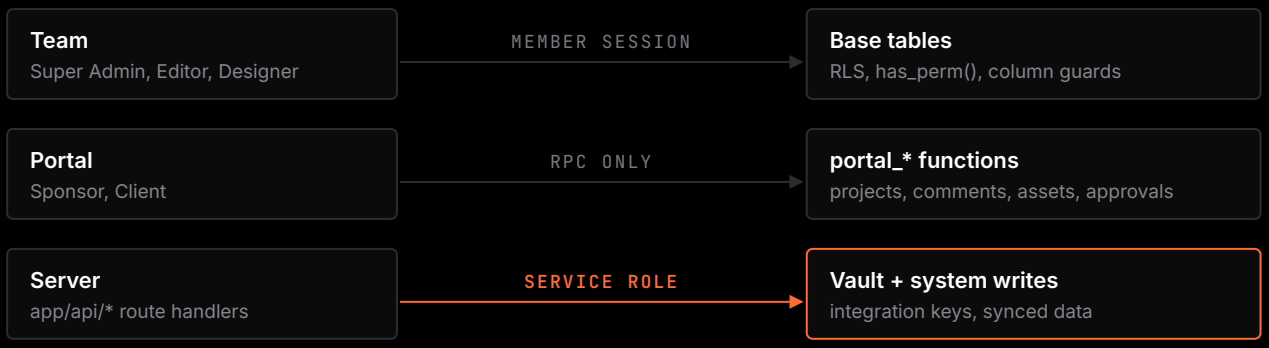
SA SUPER ADMIN · ED EDITOR · DE DESIGNER · SP SPONSOR / CLIENT

> SPONSOR LABELS
>
> Sponsors and clients see Projects as My Projects and Brand as My Brand.

06 SECURITY

WHERE RULES LIVE

Postgres is the source of truth for access.
The UI mirrors the rules for usability only.



How it holds

Row level security on every table

Roles & Permissions writes `role_permissions`. RLS reads it through `app.has_perm()`.

Assignment scope

Editors and designers read only projects they are assigned to.

Portal isolation

Sponsors and clients never read base tables. `portal_projects`, `portal_comments` and `portal_assets` return their account only.

Protected Super Admins

Any number per workspace. Only a Super Admin grants the role.
The owner and the last active Super Admin cannot be removed.

Approvals through one RPC

`portal_decide_sponsorship` records the decision.

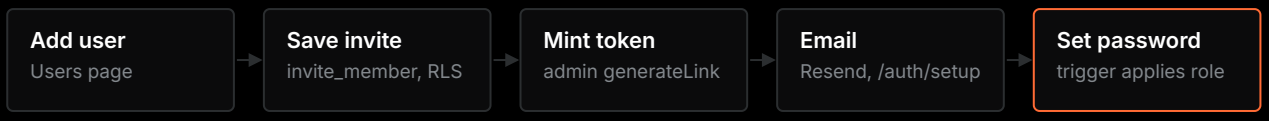
Signed-out redirect

`proxy.ts` refreshes the session and sends visitors to /login. Every API route checks its caller itself.

07 ONBOARDING

HOW PEOPLE JOIN

Admins add people from Users. The email link carries a one-time token that link scanners cannot burn.



01 **Invite is saved first.** `invite_member` runs on the admin's session, so RLS checks it.

02 **Token is minted.** The Supabase admin API creates an invite or magic link token.

03 **Email goes out.** Resend sends a link built from the token hash, never Supabase's own action link.

04 **Person sets a password.** Verifying the token confirms the email. `on_auth_user_confirmed` applies role, views and account.

Other paths

ACTION	ROUTE
Resend invite	`/api/invites/resend`
Shareable invite link	`/auth/invite/[token]` with email check
Password reset	Sign-in page

Server environment

VARIABLE	USED FOR
`SUPABASE_SECRET_KEY`	Mints the one-time token
`RESEND_API_KEY`, `EMAIL_FROM`	Sends from a verified domain

> IF EMAIL FAILS
>
> The invite still saves. The toast shows Resend's error so you can resend.

08 CONNECTED SERVICES

TEN SERVICES

Only a Super Admin connects services. Keys go to Supabase Vault and never return to the browser.

Enter key Workspace Settings → **Live check** provider verify() call → **Store** Vault, admin RPC → **Browser** sees last 4 only

SERVICE	POWERS	PAGE
Claude, OpenAI, Gemini	AI assistant. Pick a model per provider	Daily
Firecrawl	Web search and page reading	Daily, Ideas
monday.com	Two-way project and status sync, live webhooks	Projects
Apify	YouTube, TikTok, Reddit, Google scrapers	Ideas
Browserbase	Real browser on YouTube search	Ideas
Slack	Channel messages and mentions	Inbox
YouTube Data API	Uploads with views, likes, comments	Content
Buffer	Channels, sent posts, daily metrics	Content

Sync and webhooks

```
POST /api/integrations/<id>/sync  # Sync now button
Authorization: Bearer $CRON_SECRET  # scheduler, every workspace
POST /api/webhooks/monday  # live monday edits
POST /api/webhooks/slack  # live Slack events
```

Webhooks need a public `APP_BASE_URL`. On localhost, Sync now pulls the same data.

09 INTELLIGENCE

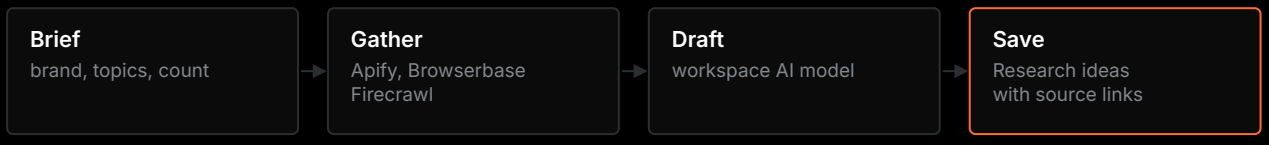
ASSISTANT + AGENT

The assistant runs its tools on the member's session.
It never does more than the member could by hand.

Assistant tools

TOOL	DOES
`search_projects`	Lists visible projects with status, dates, people, sponsor
`get_project`	One project: dates, people, distribution, comments, files
`list_ideas`	Ideas, scripts and research drafts
`plan_week`	Film, edit, thumbnail and publish dates by day
`create_project`	Proposes a project card. Saves only after you confirm
`add_idea`	Proposes an idea. Saves only after you confirm
`web_search`, `scrape_url`	Web search and page reading through Firecrawl

Research agent



Super Admin starts a run

Research button on Ideas & Research. Needs an AI provider and one source.

Runs persist

Stored in `research_runs`. They survive a reload and can be cancelled.

10 SETUP

RUN IT LOCALLY

Install, add environment keys, start. There is no demo data: every page reads the live workspace.

```
pnpm install
pnpm dev
```

Environment

VARIABLE	PURPOSE
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL
`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`	Browser key
`SUPABASE_SECRET_KEY`	Server only. Invites, Vault, synced writes
`RESEND_API_KEY`, `EMAIL_FROM`	Invite email and verified sender
`APP_BASE_URL`	Public URL for links and monday webhooks
`CRON_SECRET`	Lets a scheduler run syncs
`MONDAY_WEBHOOK_SECRET`	Signs monday webhook URLs

Not built yet

Automation runner

pg_cron, pgmq and an Edge Function. Automations stores config only today.

OAuth connectors

YouTube retention and impressions, Slack install, Drive, Gmail, Frame.io, n8n.

Two-step sign in, tests, templates

TOTP enrollment, pgTAP and Playwright suites, brand kit and template editing.